ADMINISTRATOR GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Audience**: System Administrators, IT Operations, Platform Administrators

---

TABLE OF CONTENTS

1. System Overview
2. Infrastructure Management
3. Kubernetes Administration
4. Database Administration
5. User Management
6. Monitoring & Alerting
7. Backup & Recovery
8. Security Administration
9. Troubleshooting Guide
10. Maintenance Procedures

---

1. SYSTEM OVERVIEW

1.1 Architecture Summary

```
PRODUCTION ENVIRONMENT
======================

Edge Layer (20 nodes)
  - NVIDIA Jetson AGX Orin 64GB
  - Ubuntu 22.04 + JetPack 6.0
  - Docker + TensorRT

Platform Layer (Kubernetes Cluster)
  - 3x Master Nodes (HA)
  - 4x GPU Worker Nodes (A100)
  - 6x CPU Worker Nodes

Data Layer
  - PostgreSQL 16 (3-node cluster)
  - InfluxDB 2.7 (3-node cluster)
  - Milvus 2.3 (3-node cluster)
  - Redis 7.2 (Sentinel HA)
  - MinIO (distributed mode)

Network
  - MPLS/VPN to field sites
  - DMZ for external access
  - Internal VLAN segmentation
```

1.2 Key Endpoints

Service	Internal URL	External URL	Port
API Gateway	api.oilgas.internal	api.oilgas.com	443
Dashboard	dashboard.oilgas.internal	dashboard.oilgas.com	443
Grafana	grafana.oilgas.internal	-	3000
Prometheus	prometheus.oilgas.internal	-	9090
Kibana	kibana.oilgas.internal	-	5601
Keycloak	auth.oilgas.internal	auth.oilgas.com	443
ArgoCD	argocd.oilgas.internal	-	443
MLflow	mlflow.oilgas.internal	-	5000

1.3 Admin Access

```
# SSH to bastion host
ssh admin@bastion.oilgas.internal

# Access Kubernetes
export KUBECONFIG=/etc/kubernetes/admin.conf
kubectl get nodes

# Access via VPN
# 1. Connect to Cisco AnyConnect: vpn.oilgas.com
# 2. Use AD credentials
# 3. Select "IT-Admin" profile
```

---

2. INFRASTRUCTURE MANAGEMENT

2.1 Server Inventory

Master Nodes

Hostname	IP Address	Role	Specs
k8s-master-01	10.0.1.11	Control Plane	32 CPU, 128GB RAM, 500GB SSD
k8s-master-02	10.0.1.12	Control Plane	32 CPU, 128GB RAM, 500GB SSD
k8s-master-03	10.0.1.13	Control Plane	32 CPU, 128GB RAM, 500GB SSD

GPU Worker Nodes

Hostname	IP Address	GPUs	Specs
k8s-gpu-01	10.0.2.21	2x A100 80GB	64 CPU, 512GB RAM, 2TB NVMe
k8s-gpu-02	10.0.2.22	2x A100 80GB	64 CPU, 512GB RAM, 2TB NVMe
k8s-gpu-03	10.0.2.23	2x A100 80GB	64 CPU, 512GB RAM, 2TB NVMe
k8s-gpu-04	10.0.2.24	2x A100 80GB	64 CPU, 512GB RAM, 2TB NVMe

CPU Worker Nodes

Hostname	IP Address	Role	Specs
k8s-worker-01	10.0.2.31	General workloads	32 CPU, 128GB RAM, 1TB SSD
k8s-worker-02	10.0.2.32	General workloads	32 CPU, 128GB RAM, 1TB SSD
k8s-worker-03	10.0.2.33	General workloads	32 CPU, 128GB RAM, 1TB SSD
k8s-worker-04	10.0.2.34	Database workloads	32 CPU, 256GB RAM, 2TB NVMe
k8s-worker-05	10.0.2.35	Database workloads	32 CPU, 256GB RAM, 2TB NVMe
k8s-worker-06	10.0.2.36	Database workloads	32 CPU, 256GB RAM, 2TB NVMe

Edge Nodes

Location	Hostname	IP Address	Status Check
Refinery A	edge-refa-01	10.10.1.10	`ssh edge@10.10.1.10 'systemctl status inference'`
Refinery A	edge-refa-02	10.10.1.11	`ssh edge@10.10.1.11 'systemctl status inference'`
Refinery B	edge-refb-01	10.10.2.10	`ssh edge@10.10.2.10 'systemctl status inference'`
Offshore A	edge-offa-01	10.10.3.10	`ssh edge@10.10.3.10 'systemctl status inference'`

2.2 Storage Management

Ceph Storage Cluster

```
# Check cluster health
ceph -s

# Check OSD status
ceph osd tree

# Check pool usage
ceph df

# Expected output:
# HEALTH_OK
# 12 OSDs, all up and in
# 3 pools, 50TB used of 200TB
```

Storage Classes

Name	Provisioner	Reclaim	Use Case
ceph-ssd	ceph-rbd	Retain	Databases
ceph-hdd	ceph-rbd	Delete	Logs, backups
nfs-shared	nfs-client	Retain	Shared files

```
# List storage classes
kubectl get sc

# Check PVC usage
kubectl get pvc -A | grep -v Bound

# Expand PVC (if supported)
kubectl patch pvc <pvc-name> -n <namespace> -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}
```

2.3 Network Management

VLAN Configuration

VLAN ID	Name	Subnet	Purpose
10	Management	10.0.1.0/24	Admin access
20	Kubernetes	10.0.2.0/24	K8s nodes
30	Database	10.0.3.0/24	DB traffic
40	Edge	10.10.0.0/16	Edge nodes
50	DMZ	10.0.5.0/24	External access

Firewall Rules

```
# Check firewall status
firewall-cmd --list-all

# Key ports:
# 6443 - Kubernetes API
# 443 - HTTPS services
# 9092 - Kafka
# 8086 - InfluxDB
# 5432 - PostgreSQL
# 19530 - Milvus
```

---

3. KUBERNETES ADMINISTRATION

3.1 Cluster Health Checks

Daily Health Check Script

```
#!/bin/bash
# /opt/scripts/k8s-health-check.sh

echo "=== Kubernetes Cluster Health Check ==="
echo "Date: $(date)"
echo ""
```

```
# Node status
echo "=== Node Status ==="
kubectl get nodes -o wide

# Pod issues
echo ""
echo "=== Pods Not Running ==="
kubectl get pods -A | grep -v Running | grep -v Completed

# Resource usage
echo ""
echo "=== Node Resource Usage ==="
kubectl top nodes

# PVC issues
echo ""
echo "=== PVC Not Bound ==="
kubectl get pvc -A | grep -v Bound

# Certificate expiry
echo ""
echo "=== Certificate Expiry ==="
kubeadm certs check-expiration

# Etcd health
echo ""
echo "=== Etcd Health ==="
kubectl exec -n kube-system etcd-k8s-master-01 -- etcdctl endpoint health

echo ""
echo "=== Health Check Complete ==="
```

Run Health Check

```
# Execute daily check
/opt/scripts/k8s-health-check.sh | tee /var/log/k8s-health-$(date +%Y%m%d).log

# Send to monitoring
/opt/scripts/k8s-health-check.sh | mail -s "K8s Health Report" admin@oilgas.com
```

3.2 Namespace Management

Production Namespaces

Namespace	Purpose	Resource Quota
oilgas-ai	Main application	100 CPU, 400Gi RAM
oilgas-ml	ML workloads	50 CPU, 200Gi RAM, 8 GPU
oilgas-data	Data pipelines	30 CPU, 100Gi RAM
monitoring	Prometheus, Grafana	20 CPU, 50Gi RAM
logging	ELK Stack	30 CPU, 100Gi RAM
kafka	Kafka cluster	40 CPU, 150Gi RAM

Create Namespace with Quota

```yaml
# namespace-template.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: oilgas-ai
  labels:
    environment: production
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: oilgas-ai
spec:
  hard:
    requests.cpu: "50"
    requests.memory: 200Gi
    limits.cpu: "100"
    limits.memory: 400Gi
    persistentvolumeclaims: "50"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: oilgas-ai
spec:
  limits:
  - default:
      cpu: "1"
      memory: "2Gi"
    defaultRequest:
      cpu: "100m"
      memory: "256Mi"
    type: Container
```

3.3 Deployment Management

List All Deployments

```bash
# All deployments with replicas
kubectl get deployments -A -o wide

# Deployments not fully available
kubectl get deployments -A | awk '$3 != $4 {print}'

# Deployment details
kubectl describe deployment <name> -n <namespace>
```

Scale Deployment

```bash
# Scale up
kubectl scale deployment api-gateway --replicas=5 -n oilgas-ai
```

```
# Scale down (maintenance)
kubectl scale deployment ml-inference --replicas=0 -n oilgas-ml

# Auto-scaling
kubectl autoscale deployment api-gateway --min=3 --max=10 --cpu-percent=70 -n oilgas-ai
```

Rolling Restart

```
# Restart deployment (rolling)
kubectl rollout restart deployment api-gateway -n oilgas-ai

# Check rollout status
kubectl rollout status deployment api-gateway -n oilgas-ai

# Rollback if issues
kubectl rollout undo deployment api-gateway -n oilgas-ai
```

3.4 Pod Management

Common Pod Operations

```
# List pods with issues
kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded

# Get pod logs
kubectl logs <pod-name> -n <namespace> --tail=100

# Get previous container logs (after crash)
kubectl logs <pod-name> -n <namespace> --previous

# Stream logs
kubectl logs -f <pod-name> -n <namespace>

# Execute command in pod
kubectl exec -it <pod-name> -n <namespace> -- /bin/bash

# Copy files from pod
kubectl cp <namespace>/<pod-name>:/path/to/file ./local-file

# Delete stuck pod
kubectl delete pod <pod-name> -n <namespace> --force --grace-period=0
```

Debug Pod Issues

```
# Pod events
kubectl describe pod <pod-name> -n <namespace> | grep -A 20 Events

# Resource issues
kubectl top pod <pod-name> -n <namespace>

# Network issues
kubectl exec -it <pod-name> -n <namespace> -- curl -v http://service-name:port
```

```
# DNS issues
kubectl exec -it <pod-name> -n <namespace> -- nslookup service-name
```

---

4. DATABASE ADMINISTRATION

4.1 PostgreSQL Administration

Connection Details

```
# Primary node
Host: pg-primary.oilgas.internal
Port: 5432
Database: oilgas_ai
User: oilgas_admin
Password: (in Vault: secret/data/postgres/admin)

# Connection string
psql "host=pg-primary.oilgas.internal port=5432 dbname=oilgas_ai user=oilgas_admin sslmode=require"
```

Health Checks

```
-- Check replication status
SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;

-- Check database sizes
SELECT datname, pg_size_pretty(pg_database_size(datname)) as size
FROM pg_database ORDER BY pg_database_size(datname) DESC;

-- Check table sizes
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) as size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC
LIMIT 20;

-- Active connections
SELECT datname, usename, client_addr, state, query_start, query
FROM pg_stat_activity
WHERE state = 'active';

-- Long running queries
SELECT pid, now() - pg_stat_activity.query_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle' AND now() - pg_stat_activity.query_start > interval '5 minutes';
```

Maintenance Tasks

```
# Daily vacuum (scheduled via cron)
vacuumdb --analyze --verbose oilgas_ai

# Reindex (weekly, during maintenance window)
```

```
reindexdb --verbose oilgas_ai

# Backup
pg_dump -Fc -h pg-primary.oilgas.internal -U oilgas_admin oilgas_ai > /backup/pg_backup_$(date +%Y%m%d)

# Restore
pg_restore -h pg-primary.oilgas.internal -U oilgas_admin -d oilgas_ai /backup/pg_backup_20251210.dump
```

4.2 InfluxDB Administration

Connection Details

```
# InfluxDB URL
URL: http://influxdb.oilgas.internal:8086
Org: oilgas
Token: (in Vault: secret/data/influxdb/admin-token)

# CLI access
influx config create --config-name oilgas \
  --host-url http://influxdb.oilgas.internal:8086 \
  --org oilgas \
  --token $(vault kv get -field=token secret/influxdb/admin)
```

Health Checks

```
# Check cluster health
curl -s http://influxdb.oilgas.internal:8086/health | jq

# Check bucket sizes
influx bucket list --org oilgas

# Check cardinality
influx query 'import "influxdata/influxdb" influxdb.cardinality(bucket: "sensor_data")'

# Check write throughput
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> count()
  |> group()
  |> sum()
'
```

Data Retention

```
# Check retention policies
influx bucket list --org oilgas

# Update retention
influx bucket update --id <bucket-id> --retention 7d

# Delete old data manually
influx delete --bucket sensor_data \
  --start 2024-01-01T00:00:00Z \
```

```
  --stop 2024-06-01T00:00:00Z \
  --predicate '_measurement="old_measurement"'
```

4.3 Milvus Administration

Connection Details

```
# Milvus endpoint
Host: milvus.oilgas.internal
Port: 19530
```

Health Checks

```
# Python script for Milvus health
from pymilvus import connections, utility

connections.connect(host="milvus.oilgas.internal", port="19530")

# List collections
print("Collections:", utility.list_collections())

# Check collection stats
for coll_name in utility.list_collections():
    coll = Collection(coll_name)
    print(f"{coll_name}: {coll.num_entities} entities")
```

Maintenance

```
# Compact collection
from pymilvus import Collection
coll = Collection("document_embeddings")
coll.compact()

# Rebuild index
coll.release()
coll.drop_index()
coll.create_index(
    field_name="embedding",
    index_params={"metric_type": "COSINE", "index_type": "IVF_PQ", "params": {"nlist": 1024}}
)
coll.load()
```

4.4 Redis Administration

Connection Details

```
# Redis Sentinel
redis-cli -h redis-sentinel.oilgas.internal -p 26379

# Get master
SENTINEL get-master-addr-by-name mymaster

# Connect to master
redis-cli -h <master-ip> -p 6379 -a <password>
```

Health Checks

```
# Check sentinel status
redis-cli -h redis-sentinel.oilgas.internal -p 26379 SENTINEL masters

# Memory usage
redis-cli INFO memory

# Client connections
redis-cli CLIENT LIST

# Key count
redis-cli DBSIZE
```

---

5. USER MANAGEMENT

5.1 Keycloak Administration

Access Keycloak Admin Console

```
URL: https://auth.oilgas.internal/admin
Realm: oilgas
Admin User: admin (password in Vault)
```

User Roles

Role	Permissions	Assigned To
admin	Full system access	IT Admins
operator	Dashboard, acknowledge alerts	Control Room
engineer	Dashboard, RAG, predictions	Reliability Engineers
viewer	Read-only dashboard	Management
ml-admin	MLflow, model management	Data Scientists

Create User via CLI

```
# Get admin token
TOKEN=$(curl -s -X POST "https://auth.oilgas.internal/realms/master/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=admin" \
  -d "password=$ADMIN_PASSWORD" \
  -d "grant_type=password" \
  -d "client_id=admin-cli" | jq -r '.access_token')

# Create user
curl -s -X POST "https://auth.oilgas.internal/admin/realms/oilgas/users" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "username": "john.doe",
    "email": "john.doe@oilgas.com",
    "enabled": true,
    "firstName": "John",
```

```
    "lastName": "Doe",
    "credentials": [{"type": "password", "value": "TempPass123!", "temporary": true}]
  }'

# Assign role
USER_ID=$(curl -s "https://auth.oilgas.internal/admin/realms/oilgas/users?username=john.doe" \
  -H "Authorization: Bearer $TOKEN" | jq -r '.[0].id')

curl -s -X POST "https://auth.oilgas.internal/admin/realms/oilgas/users/$USER_ID/role-mappings/realm" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '[{"name": "operator"}]'
```

Disable User

```
curl -s -X PUT "https://auth.oilgas.internal/admin/realms/oilgas/users/$USER_ID" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"enabled": false}'
```

5.2 Service Accounts

Service Account	Namespace	Purpose
api-gateway-sa	oilgas-ai	API Gateway access
ml-inference-sa	oilgas-ml	Model inference
data-pipeline-sa	oilgas-data	Kafka, databases
backup-sa	backup	Backup operations

Create Service Account

```
kubectl create serviceaccount ml-inference-sa -n oilgas-ml

# Create token (for external use)
kubectl create token ml-inference-sa -n oilgas-ml --duration=8760h
```

---

6. MONITORING & ALERTING

6.1 Prometheus

Access

```
URL: http://prometheus.oilgas.internal:9090
```

Key Metrics to Monitor

```
# Node CPU usage
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Node memory usage
(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100

# Pod restart count
```

```
sum(increase(kube_pod_container_status_restarts_total[1h])) by (namespace, pod)

# API latency P95
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Kafka consumer lag
kafka_consumer_group_lag

# InfluxDB write rate
rate(influxdb_write_points_total[5m])

# Model inference latency
histogram_quantile(0.99, rate(model_inference_duration_seconds_bucket[5m]))
```

6.2 Grafana Dashboards

Available Dashboards

Dashboard	URL	Purpose
Cluster Overview	/d/cluster	K8s cluster health
Application	/d/app	API, services metrics
ML Pipeline	/d/ml	Model performance
Data Pipeline	/d/data	Kafka, databases
Edge Nodes	/d/edge	Edge node status
Alerts	/d/alerts	Alert history

Create Dashboard Alert

```yaml
# In Grafana UI or via API
alert:
  name: High API Latency
  conditions:
    - evaluator:
        type: gt
        params: [0.5]   # 500ms
      operator:
        type: and
      query:
        params: [A, 5m, now]
      reducer:
        type: avg
  frequency: 1m
  for: 5m
  notifications:
    - uid: slack-channel
    - uid: email-oncall
```

6.3 Alert Rules

Critical Alerts

Alert	Condition	Action
NodeDown	Node unreachable 5m	Page on-call
PodCrashLooping	>5 restarts/hour	Page on-call
DiskSpaceLow	<10% free	Email + Slack
HighCPU	>90% for 15m	Slack
HighMemory	>85% for 15m	Slack
APILatencyHigh	P95 > 1s	Slack
KafkaLagHigh	>10000 messages	Email
EdgeNodeOffline	No heartbeat 10m	Page on-call

PagerDuty Integration

```yaml
# alertmanager.yml
receivers:
- name: 'pagerduty-critical'
  pagerduty_configs:
  - service_key: '<pagerduty-key>'
    severity: critical

- name: 'slack-warning'
  slack_configs:
  - api_url: '<slack-webhook>'
    channel: '#oilgas-alerts'
    title: '{{ .GroupLabels.alertname }}'
    text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'

route:
  receiver: 'slack-warning'
  routes:
  - match:
      severity: critical
    receiver: 'pagerduty-critical'
```

---

7. BACKUP & RECOVERY

7.1 Backup Schedule

Component	Frequency	Retention	Location
PostgreSQL	Hourly incremental, Daily full	30 days	S3 + DR site
InfluxDB	Continuous replication	7 years	DR site
Milvus	Daily	30 days	S3
MinIO	Daily sync	7 years	DR site
Kubernetes	etcd: hourly, configs: on change	90 days	S3
Secrets	On change	Forever	Vault + offline

7.2 Backup Commands

PostgreSQL Backup

```bash
# Manual backup
pg_dump -Fc -h pg-primary.oilgas.internal -U oilgas_admin oilgas_ai \
```

```
  | gzip > /backup/postgres/oilgas_ai_$(date +%Y%m%d_%H%M%S).dump.gz

# Upload to S3
aws s3 cp /backup/postgres/oilgas_ai_*.dump.gz s3://oilgas-backups/postgres/

# Automated backup (cron)
0 * * * * /opt/scripts/backup-postgres.sh >> /var/log/backup-postgres.log 2>&1
```

InfluxDB Backup

```
# Backup specific bucket
influx backup /backup/influxdb/$(date +%Y%m%d) \
  --host http://influxdb.oilgas.internal:8086 \
  --token $INFLUX_TOKEN \
  --bucket sensor_data

# Upload to S3
aws s3 sync /backup/influxdb/ s3://oilgas-backups/influxdb/
```

Kubernetes Backup

```
# Backup etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd/snapshot_$(date +%Y%m%d).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Backup all resources
kubectl get all --all-namespaces -o yaml > /backup/k8s/all-resources_$(date +%Y%m%d).yaml

# Backup with Velero
velero backup create daily-backup --include-namespaces oilgas-ai,oilgas-ml
```

7.3 Recovery Procedures

PostgreSQL Recovery

```
# Stop application pods
kubectl scale deployment --all --replicas=0 -n oilgas-ai

# Restore from backup
pg_restore -h pg-primary.oilgas.internal -U oilgas_admin -d oilgas_ai -c \
  /backup/postgres/oilgas_ai_20251210.dump

# Restart applications
kubectl scale deployment --all --replicas=1 -n oilgas-ai
```

Kubernetes Disaster Recovery

```
# Restore etcd from snapshot
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd/snapshot_20251210.db \
  --data-dir=/var/lib/etcd-restored

# Move to etcd data directory
```

```
systemctl stop etcd
mv /var/lib/etcd /var/lib/etcd-old
mv /var/lib/etcd-restored /var/lib/etcd
systemctl start etcd

# Restore with Velero
velero restore create --from-backup daily-backup
```

---

8. SECURITY ADMINISTRATION

8.1 Certificate Management

List Certificates

```
# Kubernetes certificates
kubeadm certs check-expiration

# Application certificates
kubectl get certificates -A

# Check specific certificate
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout | grep -A2 Validity
```

Renew Certificates

```
# Renew all K8s certificates
kubeadm certs renew all

# Restart control plane
systemctl restart kubelet

# Renew with cert-manager
kubectl cert-manager renew --all -n oilgas-ai
```

8.2 Secret Management (Vault)

Access Vault

```
# Login
vault login -method=ldap username=admin

# List secrets
vault kv list secret/oilgas

# Read secret
vault kv get secret/oilgas/postgres

# Update secret
vault kv put secret/oilgas/postgres username=admin password=NewPass123!
```

Rotate Database Passwords

```
# 1. Generate new password
NEW_PASS=$(openssl rand -base64 24)
```

```
# 2. Update in database
psql -h pg-primary.oilgas.internal -U postgres -c "ALTER USER oilgas_admin PASSWORD '$NEW_PASS'"

# 3. Update in Vault
vault kv put secret/oilgas/postgres username=oilgas_admin password=$NEW_PASS

# 4. Restart applications to pick up new credentials
kubectl rollout restart deployment -n oilgas-ai
```

8.3 Security Scanning

```
# Scan container images
trivy image oilgas/api-gateway:latest

# Scan Kubernetes
kubesec scan deployment.yaml

# Network policy audit
kubectl get networkpolicies -A

# RBAC audit
kubectl auth can-i --list --as=system:serviceaccount:oilgas-ai:api-gateway-sa
```

---

9. TROUBLESHOOTING GUIDE

9.1 Common Issues

Issue: Pods Stuck in Pending

```
# Check events
kubectl describe pod <pod-name> -n <namespace> | grep -A10 Events

# Common causes:
# 1. Insufficient resources
kubectl describe nodes | grep -A5 "Allocated resources"

# 2. PVC not bound
kubectl get pvc -n <namespace>

# 3. Node selector/affinity issues
kubectl get pod <pod-name> -n <namespace> -o yaml | grep -A10 nodeSelector
```

Issue: API Gateway 503 Errors

```
# Check API gateway pods
kubectl get pods -l app=api-gateway -n oilgas-ai

# Check logs
kubectl logs -l app=api-gateway -n oilgas-ai --tail=100

# Check upstream services
kubectl get endpoints -n oilgas-ai
```

```
# Check ingress
kubectl describe ingress api-gateway -n oilgas-ai
```

Issue: Database Connection Errors

```
# Test connectivity
kubectl run -it --rm debug --image=postgres:16 --restart=Never -- \
  psql -h pg-primary.oilgas.internal -U oilgas_admin -d oilgas_ai -c "SELECT 1"

# Check connection pool
kubectl logs <api-pod> -n oilgas-ai | grep -i connection

# Check database max connections
psql -c "SHOW max_connections"
psql -c "SELECT count(*) FROM pg_stat_activity"
```

Issue: Edge Node Offline

```
# Check VPN connectivity
ping edge-refa-01.vpn.oilgas.internal

# SSH and check services
ssh edge@edge-refa-01 'systemctl status inference mqtt-client'

# Check disk space
ssh edge@edge-refa-01 'df -h'

# Check GPU
ssh edge@edge-refa-01 'nvidia-smi'

# Restart services
ssh edge@edge-refa-01 'sudo systemctl restart inference'
```

Issue: High Kafka Lag

```
# Check consumer lag
kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group ai-platform

# Check consumer health
kubectl logs -l app=flink-consumer -n oilgas-data --tail=100

# Scale consumers
kubectl scale deployment flink-consumer --replicas=5 -n oilgas-data
```

9.2 Log Analysis

```
# Application logs (last hour, errors only)
kubectl logs -l app=api-gateway -n oilgas-ai --since=1h | grep -i error

# System logs
journalctl -u kubelet --since "1 hour ago"

# Kibana query (in Kibana)
kubernetes.namespace: "oilgas-ai" AND level: "error" AND @timestamp > now-1h
```

10. MAINTENANCE PROCEDURES

10.1 Planned Maintenance Window

Pre-Maintenance Checklist

```
[ ] Notify stakeholders (48 hours ahead)
[ ] Confirm backup completed
[ ] Review change request approved
[ ] Prepare rollback plan
[ ] Scale down non-critical workloads
[ ] Put monitoring in maintenance mode
```

Maintenance Steps

```
# 1. Notify (in Slack/PagerDuty)
/maintenance start "Scheduled maintenance - K8s upgrade"

# 2. Cordon nodes (prevent new pods)
kubectl cordon k8s-worker-01

# 3. Drain node (move pods)
kubectl drain k8s-worker-01 --ignore-daemonsets --delete-emptydir-data

# 4. Perform maintenance
# ... (upgrade, patching, etc.)

# 5. Uncordon node
kubectl uncordon k8s-worker-01

# 6. Verify
kubectl get pods -A -o wide | grep k8s-worker-01

# 7. End maintenance
/maintenance end
```

10.2 Kubernetes Upgrade

```
# 1. Backup etcd
/opt/scripts/backup-etcd.sh

# 2. Upgrade control plane (one node at a time)
kubeadm upgrade plan
kubeadm upgrade apply v1.29.0

# 3. Upgrade kubelet and kubectl
apt-get update && apt-get install -y kubelet=1.29.0-00 kubectl=1.29.0-00
systemctl daemon-reload && systemctl restart kubelet

# 4. Upgrade worker nodes (rolling)
for node in k8s-worker-01 k8s-worker-02 k8s-worker-03; do
  kubectl drain $node --ignore-daemonsets
  ssh $node 'apt-get update && apt-get install -y kubelet=1.29.0-00 && systemctl restart kubelet'
```

```
  kubectl uncordon $node
  sleep 60  # Wait for pods to stabilize
done
```

10.3 Model Update Procedure

```
# 1. Upload new model to MLflow
mlflow models download -m "models:/anomaly-detector/Production" -d /tmp/new-model

# 2. Update edge nodes (rolling)
for node in edge-refa-01 edge-refa-02 edge-refb-01; do
  echo "Updating $node..."
  scp /tmp/new-model/model.engine edge@$node:/opt/models/anomaly/
  ssh edge@$node 'systemctl restart inference'
  sleep 30
  ssh edge@$node 'curl -s localhost:8080/health' | grep -q healthy && echo "OK" || echo "FAILED"
done

# 3. Verify in monitoring
# Check Grafana dashboard for inference metrics
```

APPENDIX A: QUICK REFERENCE COMMANDS

```
# Cluster status
kubectl get nodes && kubectl get pods -A | grep -v Running

# Resource usage
kubectl top nodes && kubectl top pods -A --sort-by=memory | head -20

# Recent events
kubectl get events -A --sort-by='.lastTimestamp' | tail -20

# Service endpoints
kubectl get svc,ep -n oilgas-ai

# Logs with errors
kubectl logs -l app=api-gateway -n oilgas-ai --tail=100 | grep -i error

# Database connections
psql -h pg-primary.oilgas.internal -c "SELECT count(*) FROM pg_stat_activity"

# Kafka lag
kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --all-groups

# Edge node status
for node in edge-refa-01 edge-refb-01; do echo -n "$node: "; ssh edge@$node 'systemctl is-active infere
```

APPENDIX B: CONTACT INFORMATION

Role	Name	Phone	Email
Platform Lead	TBD	+1-xxx	platform@oilgas.com
DBA Lead	TBD	+1-xxx	dba@oilgas.com
Security Lead	TBD	+1-xxx	security@oilgas.com
On-Call	Rotating	PagerDuty	oncall@oilgas.com
Vendor Support	-	-	support@vendor.com

END OF ADMINISTRATOR GUIDE